

Design and Analysis of Algorithm

Assignment - 4

Giridhar BM
ENG24CT0040

Q) Analyze why the greedy approach fails for some optimization problems.

Sol: → Greedy does not reconsider past decision.
→ It may miss global optimal solution.
→ Condition for greedy to work → Greedy choice property
→ optimal substructure

If these not satisfied → Greedy fails

Ex 1: Knapsack.

Item	weight	Value	Ratio
A	10	60	6
B	20	100	5
C	30	120	4

Capacity = 50.

Take A → value = 60

Take B → value = 100

cannot take C

Total = 160.

Optimal solution take B + C = 220 greedy fails.

→ Greedy is fast but not always correct.

→ Works only when problem satisfies specific properties

2) Compare Bellman-Ford and Dijkstra Algorithms.
Both used to find shortest paths from a source node

Features	Dijkstra	Bellman-Ford
negative edges	→ not allowed	→ allowed
negative cycle detection	→ No	→ Yes
Time Complexity	→ $O((V+E) \log V)$	→ $O(VE)$
Approach	→ Greedy	→ Dynamic programming

Guido BM
ENG24CT0040

Working

- | | |
|---|--|
| <p>Dijkstra</p> <ul style="list-style-type: none"> → Picks minimum distance nodes → Uses priority queue. | <p>Bellman-Ford</p> <ul style="list-style-type: none"> → Repeats all edges $V-1$ times. |
|---|--|

Eg: $A \rightarrow B = 4$ $A \rightarrow C = 5$ $B \rightarrow C = -3$

Dijkstra → may give wrong result.

Bellman-Ford → gives correct result.

3) Analyze Floyd-Warshall algorithm for all pairs shortest path.

Sol: → finds shortest paths between every pair of vertices.

→ Use dynamic programming: Try every node as an intermediate node

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

Step: 1) Initialize distance matrix

2) for each vertex k :

- Update all pairs (i, j) .
Time Complexity = $O(V^3)$

Initial					After			
	A	B	C			A	B	C
A	0	3	∞	$\rightarrow$	A	0	3	4
B	∞	0	1		B	3	0	1
C	2	∞	0		C	2	5	0

Gissalhan BM
EN6124CT0040

Simple but Computationally expensive
works for dense graphs.

4) Evaluate Dynamic programming Vs Greedy Approach.

Feature	Greedy	Dynamic
Approach $\rightarrow$	Local choice $\rightarrow$	Global Optimization
Memory $\rightarrow$	Low $\rightarrow$	High
Speed $\rightarrow$	Fast $\rightarrow$	Slow
Accuracy $\rightarrow$	not always optimal $\rightarrow$	Always optimal
Example $\rightarrow$	Prim, Kruskal	Knapsack, Fibona

Eg: coin change problem

coins: (1, 3, 4) Amount = 6.

Greedy $\rightarrow 4 + 1 + 1 = 3$ coins

Optimal $\rightarrow 3 + 3 = 2$ coins.

5) Analyze knapsack problem using Dynamic vs Greedy

Types of knapsack $\rightarrow$ Fractional $\rightarrow$ Greedy Work

$\rightarrow$ 0/1 $\rightarrow$ Greedy fails $\rightarrow$ use DP

$\rightarrow$ Fractional Knapsack (Greedy)

$\rightarrow$ pick items with highest value / weight ratio.

Time Complexity: $O(n \log n)$

$\rightarrow$ 0/1 Knapsack (DP)

$dp[i][w] : \max(dp[i-1][w], \text{value}_i + dp[i-1][w - \text{weight}_i])$

either include item or exclude it.

Time complexity $\rightarrow O(nw)$

Eg:

Item	Weight	Value.
1	10	60
2	20	100
3	30	120

Greedy fails
DP gives optimal

Capacity = 50

DP gives optimal = 220.

Comparison:

Features	Greedy	DP
Work for	Fractional	0/1
Accuracy	Not Always	Always
Speed	Faster.	Slower